

# SDN-Lab3

Haochen You

April 2026

## 1 实验一：自学习交换机

### 1.1 Statement of Work

在 ARPANET 1969 年四节点拓扑 (UCLA、UCSB、SRI、UTAH) 上，实现一个二层自学习交换机，替代原有的简单洪泛交换机，避免不必要的带宽消耗和通信内容泄露。

通过修改给出的代码框架来实现。

### 1.2 问题分析

简单交换机 `simple_switch` 采用洪泛策略，即对每一个收到的数据包都执行洪泛，将其广播至交换机的所有端口。这样的策略存在两个明显的问题：

第一，带宽浪费。每次通信都向所有端口广播，主机会收到大量与自身无关的数据包；

第二，安全隐患。通信内容对网络中所有主机可见，无法保障数据的隐私性。实验中通过 Wireshark 抓包可以直接验证这一点：h1 与 h3 通信时，h4 同样能截获所有报文。

解决问题的思路：

解决上述问题的关键在于让控制器能够知道每台主机连接在哪个端口上，从而实现单播转发。之后可能会拓展出一系列的问题：数据结构如何设计？在什么时候进行映射表的学习？该如何进行转发？等等。

### 1.3 分析代码框架需要修改的内容

代码框架已经完成了控制器基础结构的搭建，下面我们分析一下我们需要增补什么：

首先，我们需要为每台交换机维护一张 MAC 地址到端口号的映射表。

其次，我们需要实现交换机的自学习与转发逻辑，具体有下面几点：

1. (映射表初始化) 当控制器首次收到某台交换机的消息时，为该交换机初始化一个空映射表。
2. (地址学习) 将数据包的源 MAC 地址与其入端口的对应关系写入该交换机的映射表，完成一次地址学习。
3. (转发端口决策) 查询目标 MAC 地址是否已存在于映射表中。若已知，则取出对应端口执行单播转发；若未知则执行洪泛。
4. (流表下发) 当目的端口已知时，调用 `add_flow` 向交换机下发 `OFPFlowMod` 消息，避免以后同样的数据包每次都要来问控制器。

5. (数据包转发) 由于流表下发的规则只对将来到达交换机的包有效, 所以我们需要构造并发送 OFPPacketOut 消息, 完成当前数据包的实际转发。

## 1.4 修改控制器代码

我们需要修改两个函数, 首先, 在 `__init__` 里加上一行, 用于初始化映射表。:

```
1 self.mac_to_port = {}
```

以及, 在 `packet_in_handler` 末尾加上下面的代码, 实现交换机的自学习与转发逻辑:

```
1 # 1. 映射表初始化
2 self.mac_to_port.setdefault(dpid, {})
3 # 2. 地址学习
4 self.mac_to_port[dpid][src] = in_port
5 # 3. 转发端口决策
6 if dst in self.mac_to_port[dpid]:
7     out_port = self.mac_to_port[dpid][dst] # 单播
8 else:
9     out_port = ofp.OFPP_FLOOD # 洪泛
10 # 4. 流表下发
11 actions = [parser.OFPAActionOutput(out_port)]
12 if out_port != ofp.OFPP_FLOOD:
13     match = parser.OFPMatch(in_port=in_port, eth_dst=dst, eth_src=src)
14     self.add_flow(dp, priority=1, match=match, actions=actions,
15                  idle_timeout=20, hard_timeout=60)
16 # 5. 当前数据包转发
17 out = parser.OFPPacketOut(
18     datapath=dp,
19     buffer_id=ofp.OFP_NO_BUFFER,
20     in_port=in_port,
21     actions=actions,
22     data=msg.data
23 )
24 dp.send_msg(out)
```

## 1.5 测试

我们先运行样例的 Controller, 验证网络能否成功建立并 ping 通, 结果如下:

```
mininet> pingall
*** Ping: testing ping reachability
SRI -> UCLA UCSB UTAH
UCLA -> SRI UCSB UTAH
UCSB -> SRI UCLA UTAH
UTAH -> SRI UCLA UCSB
*** Results: 0% dropped (12/12 received)

[sdn@archlinux sdn-lab3]$ uv run -m controllers.simple_switch
loading app controllers.simple_switch
loading app os_ken.controller.ofp_handler
instantiating app controllers.simple_switch of L2Switch
instantiating app os_ken.controller.ofp_handler of OFPHandler
```

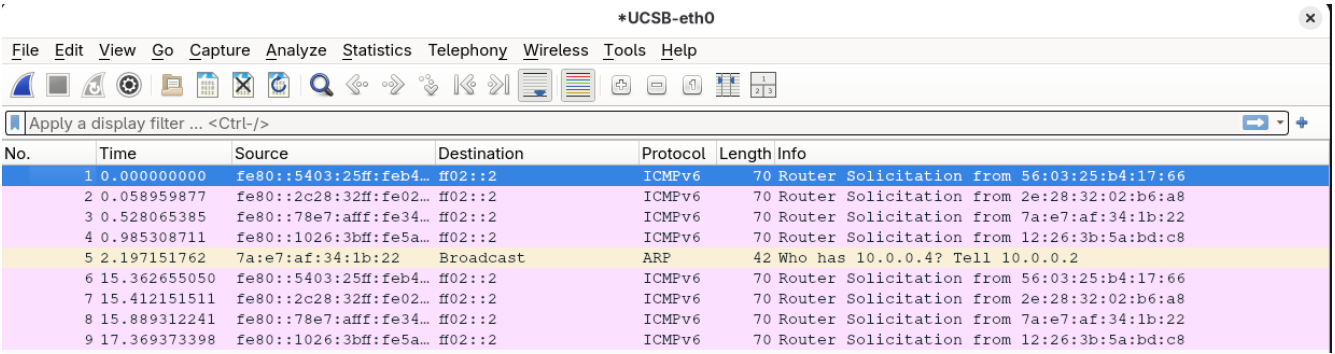
图 1: Connectivity Test

接下来我们运行我们修改过的 Controller，结果如下：

```
[sdn@archlinux sdn-lab3]$ uv run -m controllers.task1.self_learning_switch
loading app controllers.task1.self_learning_switch
loading app os_ken.controller.ofp_handler
instantiating app controllers.task1.self_learning_switch of Switch
instantiating app os_ken.controller.ofp_handler of OFPHandler
packet: 1 56:b6:fc:21:b7:0b 2a:99:bb:e2:a8:03 1
packet: 4 56:b6:fc:21:b7:0b 2a:99:bb:e2:a8:03 2
packet: 4 2a:99:bb:e2:a8:03 56:b6:fc:21:b7:0b 1
packet: 1 2a:99:bb:e2:a8:03 56:b6:fc:21:b7:0b 4
mininet> pingall
*** Ping: testing ping reachability
SRI -> UCLA UCSB UTAH
UCLA -> SRI UCSB UTAH
UCSB -> SRI UCLA UTAH
UTAH -> SRI UCLA UCSB
*** Results: 0% dropped (12/12 received)
mininet>
```

图 2: Connectivity Test

可以看到，也是能够 ping 通的。下面我们再抓包验证一下。我们在 UCSB 上抓包，然后执行 UCLA ping -c 5 UTAH，观察抓包结果：



| No. | Time         | Source                          | Destination | Protocol | Length | Info                                       |
|-----|--------------|---------------------------------|-------------|----------|--------|--------------------------------------------|
| 1   | 0.000000000  | fe80::5403:25ff:feb4... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 56:03:25:b4:17:66 |
| 2   | 0.058959877  | fe80::2c28:32ff:fe02... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 2e:28:32:02:b6:a8 |
| 3   | 0.528065385  | fe80::78e7:afff:fe34... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 7a:e7:af:34:1b:22 |
| 4   | 0.985308711  | fe80::1026:3bff:fe5a... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 12:26:3b:5a:bd:c8 |
| 5   | 2.197151762  | 7a:e7:af:34:1b:22               | Broadcast   | ARP      | 42     | Who has 10.0.0.4? Tell 10.0.0.2            |
| 6   | 15.362655050 | fe80::5403:25ff:feb4... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 56:03:25:b4:17:66 |
| 7   | 15.412151511 | fe80::2c28:32ff:fe02... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 2e:28:32:02:b6:a8 |
| 8   | 15.889312241 | fe80::78e7:afff:fe34... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 7a:e7:af:34:1b:22 |
| 9   | 17.369373398 | fe80::1026:3bff:fe5a... #f02::2 | #f02::2     | ICMPv6   | 70     | Router Solicitation from 12:26:3b:5a:bd:c8 |

图 3: Packet capture result

我们看到，只抓取到了一条 ARP 广播报文。这是 UCLA (10.0.0.2) 第一次 ping UTAH (10.0.0.4) 前发出的 ARP 广播，询问 UTAH 的 MAC 地址。而且，整个抓包过程中，UCSB 没有收到任何 ICMP 报文，即 UCLA 与 UTAH 之间的 ping 通信内容完全没有出现在 UCSB 的抓包结果中。这证明控制器在 ARP 完成后，成功学习到 UCLA 和 UTAH 各自连接的交换机端口，之后后续 ICMP 数据包实现了单播转发，不再洪泛至无关端口。

下面我们再看看 S1 上的流表：

```
[sdn@archlinux ~]$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=20.886s, table=0, n_packets=3, n_bytes=238, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=7a:e7:af:34:1b:22, dl_dst=56:03:25:b4:17:66 actions=output:"s1-eth1"
cookie=0x0, duration=20.886s, table=0, n_packets=2, n_bytes=140, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=56:03:25:b4:17:66, dl_dst=7a:e7:af:34:1b:22 actions=output:"s1-eth4"
cookie=0x0, duration=20.772s, table=0, n_packets=3, n_bytes=238, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth3", dl_src=12:26:3b:5a:bd:c8, dl_dst=56:03:25:b4:17:66 actions=output:"s1-eth1"
cookie=0x0, duration=20.771s, table=0, n_packets=2, n_bytes=140, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=56:03:25:b4:17:66, dl_dst=12:26:3b:5a:bd:c8 actions=output:"s1-eth3"
cookie=0x0, duration=20.563s, table=0, n_packets=3, n_bytes=238, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=2e:28:32:02:b6:a8, dl_dst=56:03:25:b4:17:66 actions=output:"s1-eth1"
cookie=0x0, duration=20.562s, table=0, n_packets=2, n_bytes=140, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=56:03:25:b4:17:66, dl_dst=2e:28:32:02:b6:a8 actions=output:"s1-eth2"
cookie=0x0, duration=20.283s, table=0, n_packets=3, n_bytes=238, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth3", dl_src=12:26:3b:5a:bd:c8, dl_dst=7a:e7:af:34:1b:22 actions=output:"s1-eth4"
cookie=0x0, duration=20.253s, table=0, n_packets=2, n_bytes=140, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=7a:e7:af:34:1b:22, dl_dst=12:26:3b:5a:bd:c8 actions=output:"s1-eth3"
cookie=0x0, duration=20.124s, table=0, n_packets=3, n_bytes=182, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=7a:e7:af:34:1b:22, dl_dst=2e:28:32:02:b6:a8 actions=output:"s1-eth2"
cookie=0x0, duration=20.021s, table=0, n_packets=3, n_bytes=182, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=2e:28:32:02:b6:a8, dl_dst=7a:e7:af:34:1b:22 actions=output:"s1-eth4"
cookie=0x0, duration=19.604s, table=0, n_packets=3, n_bytes=238, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=2e:28:32:02:b6:a8, dl_dst=12:26:3b:5a:bd:c8 actions=output:"s1-eth3"
cookie=0x0, duration=19.532s, table=0, n_packets=2, n_bytes=140, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth3", dl_src=12:26:3b:5a:bd:c8, dl_dst=2e:28:32:02:b6:a8 actions=output:"s1-eth2"
cookie=0x0, duration=915.245s, table=0, n_packets=108, n_bytes=10134, priority=0 actions=CONTROLLER:65535
```

图 4: Flow Table

查询结果显示，s1 上共存在 11 条流表项，其中 10 条为控制器通过下发的精流表项，1 条为初始下发的用于兜底的 table-miss 流表项。这验证了代码的流表下发机制运行是正常的。



```
mininet> dpctl dump-flows
*** s1 ***
cookie=0x0, duration=56.287s, table=0, n_packets=275218, n_bytes=11559156, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=82:2a:a7:d5:4f:18, dl_dst=f6:5d:f6:56:be:43 actions=output:"s1-eth3"
cookie=0x0, duration=56.268s, table=0, n_packets=6519, n_bytes=273798, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=82:2a:a7:d5:4f:18, dl_dst=f6:5d:f6:56:be:43 actions=output:"s1-eth3"
cookie=0x0, duration=42.372s, table=0, n_packets=278066, n_bytes=11342772, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=22:5e:c9:3b:3c:9a, dl_dst=b2:4b:f6:ff:b1:90 actions=output:"s1-eth3"
cookie=0x0, duration=39.177s, table=0, n_packets=182421, n_bytes=7661682, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=f6:5d:f6:56:be:43, dl_dst=b2:4b:f6:ff:b1:90 actions=output:"s1-eth3"
cookie=0x0, duration=35.998s, table=0, n_packets=162252, n_bytes=6814584, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=82:2a:a7:d5:4f:18, dl_dst=b2:4b:f6:ff:b1:90 actions=output:"s1-eth3"
cookie=0x0, duration=35.972s, table=0, n_packets=4261, n_bytes=178962, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=82:2a:a7:d5:4f:18, dl_dst=b2:4b:f6:ff:b1:90 actions=output:"s1-eth3"
cookie=0x0, duration=22.848s, table=0, n_packets=92472, n_bytes=3883824, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=f6:5d:f6:56:be:43, dl_dst=22:5e:c9:3b:3c:9a actions=output:"s1-eth3"
cookie=0x0, duration=19.651s, table=0, n_packets=79795, n_bytes=3351398, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=82:2a:a7:d5:4f:18, dl_dst=22:5e:c9:3b:3c:9a actions=output:"s1-eth3"
cookie=0x0, duration=19.658s, table=0, n_packets=2331, n_bytes=97902, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=82:2a:a7:d5:4f:18, dl_dst=22:5e:c9:3b:3c:9a actions=output:"s1-eth3"
*** s2 ***
cookie=0x0, duration=56.619s, table=0, n_packets=6520, n_bytes=273840, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=82:2a:a7:d5:4f:18, dl_dst=f6:5d:f6:56:be:43 actions=output:"s2-eth2"
cookie=0x0, duration=36.821s, table=0, n_packets=2462, n_bytes=179804, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=82:2a:a7:d5:4f:18, dl_dst=b2:4b:f6:ff:b1:90 actions=output:"s2-eth2"
cookie=0x0, duration=19.773s, table=0, n_packets=2331, n_bytes=97902, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=82:2a:a7:d5:4f:18, dl_dst=22:5e:c9:3b:3c:9a actions=output:"s2-eth2"
cookie=0x0, duration=167.218s, table=0, n_packets=289437, n_bytes=11091986, priority=0 actions=CONTROLLER:65535
*** s3 ***
```

图 7: Flow Table

这是因为新拓扑在任务一的基础上，S3 和 S4 之间加了一条直连链路，网络就变成了有环图。这会导致什么呢？如果 UCLA 想 ping UTAH，第一步要发 ARP 广播报文来找 UTAH。这个广播包在有环拓扑里会这样传播：s4->s3->s1->s4……这样无限传播下去，流表匹配次数就会指数上涨，消耗 CPU 资源之余，还会导致流表被错误的映射污染。

## 2.3 修改代码框架 & 测试

本质上，我们就是要实现 ARP 的去重逻辑。即对于每台交换机，要判断出来“之前是否接收过 IP1 寻找 IP2 的广播”。如果没有接收过，就记下来；如果是，则丢弃即可。

另外，我们还要把之前自学习逻辑的部分复制过来。

控制器代码框架的增加部分如下：

```
1 arp_pkt = header_list[ARP] #目标IP
2 dst_ip = arp_pkt.dst_ip
3 # 查去重表
4 if (dpid, src, dst_ip) in self.sw:
5     if self.sw[(dpid, src, dst_ip)] != in_port: # 同一个(src_mac, dst_ip), 丢弃
6         return
7 else:
8     self.sw[(dpid, src, dst_ip)] = in_port # 第一次见, 记录
```

测试连通性：

```
sdn@archlinux:~/sdn-lab3
[sdn@archlinux ~]$ cd sdn-lab3
[sdn@archlinux sdn-lab3]$ ./run -w controllers.task2.loop_detecting_switch
loading app controller.task2.loop_detecting_switch
loading app sdn_controller.app_header
instantiating app controllers.task2.loop_detecting_switch of Switch_Sict
instantiating app sdn_controller.app_header of OFFlowCtrl
[]

sdn@archlinux:~/sdn-lab3
[sdn@archlinux ~]$ cd sdn-lab3
[sdn@archlinux sdn-lab3]$ sudo ./run topo/topo_1903_2.py
*** Creating network
*** Adding controller
*** Adding hosts
SRI UCLA UCSB UTAH
*** Adding switches
s1 s2 s3 s4
*** Adding links:
(s1, SRI) (10.0.0.0/1) (10.0.0.0/1) (s1, s2) (10.0.0.0/1) (10.0.0.0/1) (s1, s3) (10.0.0.0/1)
(10.0.0.0/1) (s1, s4) (s2, UTAH) (s3, UCSB) (s3, UCSB) (10.0.0.0/1) (10.0.0.0/1) (s3, s4) (s4, UCLA)
*** Configuring hosts
SRI UCLA UCSB UTAH
*** Starting controller
c1
*** Starting 4 switches
s1 s2 s3 s4 ... (10.0.0.0/1) (10.0.0.0/1) (10.0.0.0/1) (10.0.0.0/1) (10.0.0.0/1) (10.0.0.0/1)
(10.0.0.0/1) (10.0.0.0/1)
SRI SRI-eth0:01-eth0
UCLA UCLA-eth0:01-eth0
UCSB UCSB-eth0:01-eth0
UTAH UTAH-eth0:01-eth0
*** Starting d11
mininet> pingall
*** Ping: testing ping reachability
SRI -> UCLA UCSB UTAH
UCLA -> SRI UCSB UTAH
UCSB -> SRI UCLA UTAH
UTAH -> SRI UCLA UCSB
*** Results: 0% dropped (12/12 received)
mininet> []
```

图 8: Connection Test

已经能全部 ping 通了！之后我们再测试一下流表项，结果如下：

```
mininet> dpctl dump-flows
*** s1 ***
cookie=0x0, duration=2.618s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=da:3d:51:c6:64:4a, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s1-eth4"
cookie=0x0, duration=2.614s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=e6:b4:bb:96:40:d7, dl_dst=da:3d:51:c6:64:4a actions=output:"s1-eth1"
cookie=0x0, duration=2.808s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=da:3d:51:c6:64:4a, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s1-eth3"
cookie=0x0, duration=2.806s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth3", dl_src=ba:9b:1b:0a:72:23, dl_dst=da:3d:51:c6:64:4a actions=output:"s1-eth1"
cookie=0x0, duration=2.808s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth1", dl_src=da:3d:51:c6:64:4a, dl_dst=d2:44:a9:41:96:2f actions=output:"s1-eth2"
cookie=0x0, duration=2.598s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=d2:44:a9:41:96:2f, dl_dst=da:3d:51:c6:64:4a actions=output:"s1-eth1"
cookie=0x0, duration=2.577s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth4", dl_src=e6:b4:bb:96:40:d7, dl_dst=d2:44:a9:41:96:2f actions=output:"s1-eth2"
cookie=0x0, duration=2.575s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=d2:44:a9:41:96:2f, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s1-eth4"
cookie=0x0, duration=2.568s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth3", dl_src=ba:9b:1b:0a:72:23, dl_dst=d2:44:a9:41:96:2f actions=output:"s1-eth2"
cookie=0x0, duration=2.567s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s1-eth2", dl_src=d2:44:a9:41:96:2f, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s1-eth3"
cookie=0x0, duration=303.185s, table=0, n_packets=80, n_bytes=8178, priority=0 actions=CONTROLLER:65535
*** s2 ***
cookie=0x0, duration=2.683s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth2", dl_src=da:3d:51:c6:64:4a, dl_dst=d2:44:a9:41:96:2f actions=output:"s2-eth1"
cookie=0x0, duration=2.802s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=d2:44:a9:41:96:2f, dl_dst=da:3d:51:c6:64:4a actions=output:"s2-eth2"
cookie=0x0, duration=2.588s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth2", dl_src=e6:b4:bb:96:40:d7, dl_dst=d2:44:a9:41:96:2f actions=output:"s2-eth1"
cookie=0x0, duration=2.588s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=d2:44:a9:41:96:2f, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s2-eth2"
cookie=0x0, duration=2.572s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth2", dl_src=ba:9b:1b:0a:72:23, dl_dst=d2:44:a9:41:96:2f actions=output:"s2-eth1"
cookie=0x0, duration=2.572s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s2-eth1", dl_src=d2:44:a9:41:96:2f, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s2-eth2"
cookie=0x0, duration=303.188s, table=0, n_packets=43, n_bytes=3944, priority=0 actions=CONTROLLER:65535
*** s3 ***
cookie=0x0, duration=2.615s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth2", dl_src=da:3d:51:c6:64:4a, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s3-eth1"
cookie=0x0, duration=2.614s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth1", dl_src=ba:9b:1b:0a:72:23, dl_dst=da:3d:51:c6:64:4a actions=output:"s3-eth2"
cookie=0x0, duration=2.596s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth3", dl_src=e6:b4:bb:96:40:d7, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s3-eth1"
cookie=0x0, duration=2.596s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth1", dl_src=ba:9b:1b:0a:72:23, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s3-eth3"
cookie=0x0, duration=2.576s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth1", dl_src=ba:9b:1b:0a:72:23, dl_dst=d2:44:a9:41:96:2f actions=output:"s3-eth2"
cookie=0x0, duration=2.574s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s3-eth2", dl_src=d2:44:a9:41:96:2f, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s3-eth1"
cookie=0x0, duration=303.191s, table=0, n_packets=61, n_bytes=5866, priority=0 actions=CONTROLLER:65535
*** s4 ***
cookie=0x0, duration=2.646s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth2", dl_src=da:3d:51:c6:64:4a, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s4-eth1"
cookie=0x0, duration=2.643s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth1", dl_src=e6:b4:bb:96:40:d7, dl_dst=da:3d:51:c6:64:4a actions=output:"s4-eth2"
cookie=0x0, duration=2.618s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth1", dl_src=e6:b4:bb:96:40:d7, dl_dst=ba:9b:1b:0a:72:23 actions=output:"s4-eth3"
cookie=0x0, duration=2.615s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth3", dl_src=ba:9b:1b:0a:72:23, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s4-eth1"
cookie=0x0, duration=2.611s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth1", dl_src=e6:b4:bb:96:40:d7, dl_dst=d2:44:a9:41:96:2f actions=output:"s4-eth2"
cookie=0x0, duration=2.603s, table=0, n_packets=1, n_bytes=98, idle_timeout=20, hard_timeout=60, priority=1, in_port="s4-eth2", dl_src=d2:44:a9:41:96:2f, dl_dst=e6:b4:bb:96:40:d7 actions=output:"s4-eth1"
cookie=0x0, duration=303.213s, table=0, n_packets=59, n_bytes=5714, priority=0 actions=CONTROLLER:65535
```

图 9: Flow Table

这下完全正确。可以看到 ARP 去重是生效的，包数量正常且自学习正常。

## 3 实验三：优化路由策略

### 3.1 Statement of Work

首先我们先把新的拓扑图画出来：

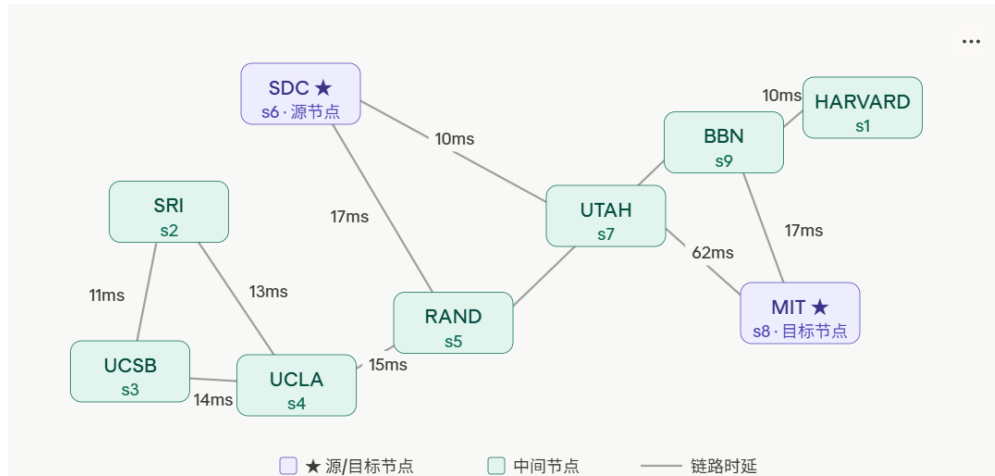


图 10: Topo Graph

因为网络变大了，简单的自学习已经不够用了。现在需要控制器主动掌握全局信息，计算出最优路径再下发。我们总结一下从热身一到完成实验的所有工作：



首先是热身一：拓扑感知，这是为了交换机能够知道网络里的拓扑情况（有几个交换机，有哪些链路等）；其次是热身二：最少跳数路径，这一步我们实现经过交换机数量最少的路径，为下一步做准备；最后是实验任务：最小时延路径。这一步我们相当于给图中的边附加上了权值。于是这就引出了一系列问题：边权时延怎么求？怎么保存？怎么求最小时延路径？等等。

下面我们一步步解决这些问题：

## 3.2 WarmUp1

我们运行 demo Controller，这个文件能帮我们打印出来观察拓扑的结果，如下图所示（只展示了交换机和主机部分）。同时，我们也能观察到指导书中写的“沉默主机”现象，'ipv4' 这里全都为空。

```
hosts:
('mac': '26:98:b8:9e:d9:44', 'ipv4': [], 'ipv6': ['::', 'fe80::2498:b8ff:fe9e:d944'], 'port': {'dpid': '0000000000000002', 'port_no': '00000001', 'hw_addr': '6e:60:bc:c5:f6:0f', 'name': 's2-eth1'})
('mac': '0a:74:e3:4c:9c:73', 'ipv4': [], 'ipv6': ['::', 'fe80::874:e3ff:fe4c:c73'], 'port': {'dpid': '0000000000000004', 'port_no': '00000001', 'hw_addr': '8a:45:e6:f5:4b:04', 'name': 's4-eth1'})
('mac': '62:b5:e4:d3:5b:98', 'ipv4': [], 'ipv6': ['::', 'fe80::60b5:e4ff:fe63:5b98'], 'port': {'dpid': '0000000000000003', 'port_no': '00000001', 'hw_addr': '0e:f8:d9:00:01:85', 'name': 's3-eth1'})
('mac': '2a:36:e4:f5:a6:e0', 'ipv4': [], 'ipv6': ['::', 'fe80::2a36:e4ff:fef5:a6e0'], 'port': {'dpid': '0000000000000001', 'port_no': '00000001', 'hw_addr': '62:fb:94:3d:a8:2c', 'name': 's1-eth1'})
('mac': 'fa:94:24:4a:42:8e', 'ipv4': [], 'ipv6': ['::', 'fe80::fa94:24ff:fe4a:428e'], 'port': {'dpid': '0000000000000005', 'port_no': '00000001', 'hw_addr': '7a:5e:11:04:4a:26', 'name': 's5-eth1'})
('mac': '8a:6a:a6:7b:27:6b', 'ipv4': [], 'ipv6': ['::', 'fe80::8a6a:a6ff:fe7b:276b'], 'port': {'dpid': '0000000000000008', 'port_no': '00000001', 'hw_addr': '8e:43:67:02:e4:d8', 'name': 's8-eth1'})
('mac': '7a:b9:23:39:39:d6', 'ipv4': [], 'ipv6': ['::', 'fe80::7ab9:23ff:fe39:39d6'], 'port': {'dpid': '0000000000000007', 'port_no': '00000001', 'hw_addr': 'b2:0e:59:4d:15:9b', 'name': 's7-eth1'})
('mac': '1e:b5:36:7f:f6:55', 'ipv4': [], 'ipv6': ['::', 'fe80::1eb5:36ff:fe7f:f655'], 'port': {'dpid': '0000000000000009', 'port_no': '00000001', 'hw_addr': '1a:1d:15:00:09:46', 'name': 's9-eth1'})
('mac': 'd8:87:39:e8:37:90', 'ipv4': [], 'ipv6': ['::', 'fe80::d887:39ff:fe88:3790'], 'port': {'dpid': '0000000000000006', 'port_no': '00000001', 'hw_addr': '8e:71:28:e4:9c:5f', 'name': 's6-eth1'})

switches:
({'dpid': '0000000000000001', 'ports': [{'dpid': '0000000000000001', 'port_no': '00000001', 'hw_addr': '62:fb:94:3d:a8:2c', 'name': 's1-eth1'}, {'dpid': '0000000000000001', 'port_no': '00000002', 'hw_addr': '72:72:8b:95:17:87', 'name': 's1-eth2'}]})
({'dpid': '0000000000000003', 'ports': [{'dpid': '0000000000000003', 'port_no': '00000002', 'hw_addr': '92:0c:51:80:2c:5c', 'name': 's3-eth2'}, {'dpid': '0000000000000003', 'port_no': '00000001', 'hw_addr': '0e:f8:d9:00:01:85', 'name': 's3-eth1'}, {'dpid': '0000000000000003', 'port_no': '00000003', 'hw_addr': 'aa:cc:96:5c:8e:00', 'name': 's3-eth3'}]})
({'dpid': '0000000000000002', 'ports': [{'dpid': '0000000000000002', 'port_no': '00000001', 'hw_addr': '6e:60:bc:c5:f6:0f', 'name': 's2-eth1'}, {'dpid': '0000000000000002', 'port_no': '00000002', 'hw_addr': '6e:25:70:03:94:11', 'name': 's2-eth2'}, {'dpid': '0000000000000002', 'port_no': '00000003', 'hw_addr': 'aa:14:f1:9e:14:4c', 'name': 's2-eth3'}]})
({'dpid': '0000000000000004', 'ports': [{'dpid': '0000000000000004', 'port_no': '00000002', 'hw_addr': '1a:39:80:ec:b2:10', 'name': 's4-eth2'}, {'dpid': '0000000000000004', 'port_no': '00000003', 'hw_addr': '5a:ic:39:fe:1e:05:c2', 'name': 's4-eth3'}, {'dpid': '0000000000000004', 'port_no': '00000001', 'hw_addr': '8a:45:e6:f5:4b:04', 'name': 's4-eth1'}, {'dpid': '0000000000000004', 'port_no': '00000004', 'hw_addr': '3a:75:8f:6b:9f:29', 'name': 's4-eth4'}]})
({'dpid': '0000000000000007', 'ports': [{'dpid': '0000000000000007', 'port_no': '00000001', 'hw_addr': 'b2:0e:59:4d:15:9b', 'name': 's7-eth1'}, {'dpid': '0000000000000007', 'port_no': '00000002', 'hw_addr': '52:f3:97:ce:a9:8e', 'name': 's7-eth2'}, {'dpid': '0000000000000007', 'port_no': '00000003', 'hw_addr': '5a:a3:c9:4c:c8:ed', 'name': 's7-eth3'}]})
({'dpid': '0000000000000006', 'ports': [{'dpid': '0000000000000006', 'port_no': '00000001', 'hw_addr': '8e:71:28:e4:9c:5f', 'name': 's6-eth1'}, {'dpid': '0000000000000006', 'port_no': '00000002', 'hw_addr': '0e:76:a8:b9:a6:33', 'name': 's6-eth2'}, {'dpid': '0000000000000006', 'port_no': '00000003', 'hw_addr': 'aa:2f:c5:cb:ae:93', 'name': 's6-eth3'}]})
({'dpid': '0000000000000009', 'ports': [{'dpid': '0000000000000009', 'port_no': '00000001', 'hw_addr': '1a:1d:15:00:09:46', 'name': 's9-eth1'}, {'dpid': '0000000000000009', 'port_no': '00000004', 'hw_addr': '0a:e3:8d:15:cd:1d', 'name': 's9-eth4'}, {'dpid': '0000000000000009', 'port_no': '00000002', 'hw_addr': '8a:30:4b:b8:84:63', 'name': 's9-eth2'}, {'dpid': '0000000000000009', 'port_no': '00000003', 'hw_addr': 'a2:6a:16:62:13:bb', 'name': 's9-eth3'}]})
({'dpid': '0000000000000008', 'ports': [{'dpid': '0000000000000008', 'port_no': '00000001', 'hw_addr': '8e:43:67:02:e4:d8', 'name': 's8-eth1'}, {'dpid': '0000000000000008', 'port_no': '00000002', 'hw_addr': '22:e9:cc:47:25:b2', 'name': 's8-eth2'}, {'dpid': '0000000000000008', 'port_no': '00000003', 'hw_addr': 'be:72:c4:76:1e:02', 'name': 's8-eth3'}]})
({'dpid': '0000000000000005', 'ports': [{'dpid': '0000000000000005', 'port_no': '00000004', 'hw_addr': '76:40:6a:98:68:8c', 'name': 's5-eth4'}, {'dpid': '0000000000000005', 'port_no': '00000001', 'hw_addr': '7a:5e:11:04:4a:26', 'name': 's5-eth1'}, {'dpid': '0000000000000005', 'port_no': '00000002', 'hw_addr': 'da:88:ff:bb:24:03', 'name': 's5-eth2'}, {'dpid': '0000000000000005', 'port_no': '00000003', 'hw_addr': 'fe:36:34:40:7e:2d', 'name': 's5-eth3'}]})
```

图 11: Topo

## 3.3 WarmUp2

在上一个任务中我们实现了“控制器能看到整个网络”，但是为了实现转发，我们的控制器肯定还要实现额外的功能才能让整个网络 ping 通。

考虑下面的过程：收到任意两台主机之间的 IPV4 包时，控制器需要查出来源 IP 和目的 IP，接下来在拓扑图上算出跳数最少的路径，之后把路径上每台交换机的转发规则转发下去，这些交换机学习之后，主机之间就能 ping 通了。我们要实现的就是上面的这些逻辑。我们发现这和之前的逻辑是有一定区别的。之前控制器发现数据包来了，就看一下数据包从哪来的，记下来之后就知道了下次怎么转发了。现在包来了需要调用 network\_awareness，询问从 A 到 B 要走多少跳，拿到完整最短路径之后把沿途所有交换机的流表都下发好。

分析完之后，我们把之前处理环路的逻辑导入到 handle\_arp 函数中，随即跑起来验证一下：

```
mininet> pingall
*** Ping: testing ping reachability
BBN -> X X X X X X X X
HARVARD -> BBN MIT RAND SDC SRI UCLA UCSB UTAH
MIT -> BBN HARVARD RAND SDC SRI UCLA UCSB UTAH
RAND -> BBN HARVARD MIT SDC SRI UCLA UCSB UTAH
SDC -> BBN HARVARD MIT RAND SRI UCLA UCSB UTAH
SRI -> BBN HARVARD MIT RAND SDC UCLA UCSB UTAH
UCLA -> BBN HARVARD MIT RAND SDC SRI UCSB UTAH
UCSB -> BBN HARVARD MIT RAND SDC SRI UCLA UTAH
UTAH -> BBN HARVARD MIT RAND SDC SRI UCLA UCSB
*** Results: 11% dropped (64/72 received)
```

图 12: Pingall(1)

```
mininet> pingall
*** Ping: testing ping reachability
BBN -> HARVARD MIT RAND SDC SRI UCLA UCSB UTAH
HARVARD -> BBN MIT RAND SDC SRI UCLA UCSB UTAH
MIT -> BBN HARVARD RAND SDC SRI UCLA UCSB UTAH
RAND -> BBN HARVARD MIT SDC SRI UCLA UCSB UTAH
SDC -> BBN HARVARD MIT RAND SRI UCLA UCSB UTAH
SRI -> BBN HARVARD MIT RAND SDC UCLA UCSB UTAH
UCLA -> BBN HARVARD MIT RAND SDC SRI UCSB UTAH
UCSB -> BBN HARVARD MIT RAND SDC SRI UCLA UTAH
UTAH -> BBN HARVARD MIT RAND SDC SRI UCLA UCSB
*** Results: 0% dropped (72/72 received)
```

图 13: Pingall(2)

可以看到，第一次执行 pingall 的时候只有 BBN 发出去的包不通，后面的包都 ping 通了。这也是因为之前提到的沉默主机现象。ping 之前控制器没学到这些主机的 IP，所以 ping 不通。后面学到这些主机的 IP 之后，再 ping 就能够通了。

### 3.4 最小时延路径

#### 3.4.1 修改 OS-Ken 源码

其实我们要实现的任务相当于进阶版的 WarmUp2，把路由依据从跳数换成测试出来的时延。控制器要测量 S1->S2 的链路时延，思路是：先发一个带时间戳的 LLDP 包给 S1，S1 转发给 S2，S2 收到后上报给控制器。时延就是收到的时间戳减去发送的时间戳。但是问题来了，OS-Ken 发送 LLDP 包的时候，没有记录时延这个属性。所以我们要对源码进行修改。按照指导书的思路，修改有以下几点：

1. 在每个记录端口信息的类里，加上 delay 属性；

修改后的部分代码：

```
1 class PortData(object):
2     def __init__(self, is_down, lldp_data):
3         super(PortData, self).__init__()
4         self.is_down = is_down
5         self.lldp_data = lldp_data
6         self.timestamp = None
7         self.sent = 0
8         self.delay = 0
```

2. 收到 LLDP 包时，计算时延并存入 LLDP 包对应的交换机端口；

修改后的部分代码：

```
1 在 lldp_packet_in_handler(self, ev) 函数中:
2     recv_timestamp = time.time()#新增
3     ... ..
4     try:
5         self.ports.lldp_received(src)
6     except KeyError:
7         # There are races between EventOFPPacketIn and
```



```

8     # EventDPPortAdd. So packet-in event can happend before
9     # port add event. In that case key error can happend.
10    # LOG.debug('lldp_received error', exc_info=True)
11    pass
12    #下面是新增部分
13    port_data = self.ports.get(src)
14    if port_data and port_data.timestamp:
15        port_data.delay = recv_timestamp - port_data.timestamp

```

### 3.4.2 编写控制器

现在 OS-Ken 能够记录每个端口的 LLDP 时延了。但光有数据还不够，我们需要编写一个新的控制器把这些 LLDP 时延读出来、算成链路时延、用于路由决策。

下面我们新建 `shortest_forward.py`，首先分析一下这个控制器要实现哪些任务，并且怎么实现：

1. 它要周期性的发送 Echo Request，测控制器到每台交换机的 RTT（用于计算链路时延）；
2. 它能周期性的读取 `lldp_delay`，并用这个数据和 Echo Reply 时间计算出链路时延；
3. 它能调用 `network_awareness.py`，重点是能调用其维护的 `topo_map` 和 `shortest_path`。

我们基于 `least_hops.py` 提供的框架，在这个框架之上增添的有如下几个部分：

在 `__init__` 里，增加新数据部分：

```

1 self.switches = None           # 存 switches 实例
2 self.echo_delay = {}          # dpid -> 控制器到该交换机的单程时延
3 self.lldp_delay = {}          # (src_dpid, dst_dpid) -> lldp时延
4 self.link_delay = {}          # (src_dpid, dst_dpid) -> 链路时延
5
6 # 启动计算时延的后台线程
7 self.echo_thread = hub.spawn(self._send_echo_request)
8 self.delay_thread = hub.spawn(self._update_link_delay)

```

加 Echo Request 的发送和接收：

```

1 # 周期性发 Echo Request
2 def _send_echo_request(self):
3     while True:
4         for dpid, dp in self.network_awareness.switch_info.items():
5             parser = dp.ofproto_parser
6             echo_req = parser.OFPEchoRequest(dp, data=str(time.time()).encode())
7             dp.send_msg(echo_req)
8             hub.sleep(0.05)
9
10    # 收到 Echo Reply 时计算 RTT
11    @set_ev_cls(ofp_event.EventOFPEchoReply, MAIN_DISPATCHER)
12    def echo_reply_handler(self, ev):
13        recv_time = time.time()
14        send_time = float(ev.msg.data.decode())
15        dpid = ev.msg.datapath.id
16        self.echo_delay[dpid] = (recv_time - send_time) / 2

```

加时延计算并更新拓扑图：

```
1 def _get_lldp_delay(self, src_dpid, dst_dpid):
2     if self.switches is None:
3         self.switches = lookup_service_brick('switches')
4     for port in self.switches.ports.keys():
5         if port.dpid == src_dpid:
6             # 确认这个端口连的就是 dst_dpid
7             link = self.network_awareness.port_link.get((src_dpid, port.port_no))
8             if link and link[1] == dst_dpid:
9                 return self.switches.ports[port].delay
10    return 0
11
12 # 周期性更新每条链路的时延到 topo_map
13 def _update_link_delay(self):
14     while True:
15         for src_dpid, dst_dpid in self.network_awareness.link_info:
16             # 跳过主机节点
17             if not isinstance(src_dpid, int) or not isinstance(dst_dpid, int):
18                 continue
19             lldp_s12 = self._get_lldp_delay(src_dpid, dst_dpid)
20             lldp_s21 = self._get_lldp_delay(dst_dpid, src_dpid)
21             echo_s1 = self.echo_delay.get(src_dpid, 0)
22             echo_s2 = self.echo_delay.get(dst_dpid, 0)
23             delay = (lldp_s12 + lldp_s21 - echo_s1 - echo_s2) / 2
24             delay = max(0, delay) # 负数取0
25             # 写进拓扑图
26             if self.network_awareness.topo_map.has_edge(src_dpid, dst_dpid):
27                 self.network_awareness.topo_map[src_dpid][dst_dpid]['delay'] = delay
28
29     hub.sleep(2)
```

经过上面这些改动，新的控制器就能实现我们想要的功能了。

### 3.4.3 测试

我们先构建拓扑，看看连通性如何：

```
mininet> pingall
*** Ping: testing ping reachability
BBN -> X X X X X X X X
HARVARD -> BBN MIT RAND SDC SRI UCLA UCSB UTAH
MIT -> BBN HARVARD RAND SDC SRI UCLA UCSB UTAH
RAND -> BBN HARVARD MIT SDC SRI UCLA UCSB UTAH
SDC -> BBN HARVARD MIT RAND SRI UCLA UCSB UTAH
SRI -> BBN HARVARD MIT RAND SDC UCLA UCSB UTAH
UCLA -> BBN HARVARD MIT RAND SDC SRI UCSB UTAH
UCSB -> BBN HARVARD MIT RAND SDC SRI UCLA UTAH
UTAH -> BBN HARVARD MIT RAND SDC SRI UCLA UCSB
*** Results: 11% dropped (64/72 received)
```

图 14: Test1

可以看到，能够 ping 通（第一次 ping 不通是因为沉默主机现象）。

接下来我们执行 SDC ping -c 1 MIT，看看路径和 time 是怎样的，下面是路径：

```
path: 10.0.0.5 -> 10.0.0.3
10.0.0.5 -> 1:s6:2 -> 4:s5:3 -> 3:s9:4 -> 3:s8:1 -> 10.0.0.3
```

图 15: Path

可以看到和指导书 ping 的路径是一样的，下面再看看 time：

```
mininet> SDC ping -c 10 MIT
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=130 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=134 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=132 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=137 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=138 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=133 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=139 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=137 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=133 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=134 ms

--- 10.0.0.3 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9028ms
rtt min/avg/max/mdev = 130.358/134.740/139.159/2.791 ms
```

图 16: Time

可以看到，时间也是符合我们测试的预期的，这说明我们的控制器在正确工作。

## 4 实验四：容忍链路故障

### 4.1 Statement of Work

任务三实现了最小时延路径，但有一个问题：如果路径上某条链路断了怎么办？现在的 `shortest_forward` 会一直用旧的流表转发，包全部丢失，直到超时才会重新计算。下面我们则要实现：链路故障时自动切换到新的最小时延路径，链路恢复后再切换回来。

### 4.2 修改控制器代码

首先，链路断了之后，链路两端交换机的端口状态会立刻变化，OS-Ken 会触发 `EventOFPPortStatus` 事件。所以我们新的控制器要有一个函数监听这个事件，才能察觉到链路出现故障。还有，我们要及时更新拓扑图里的链路，删去故障的链路，并在 OVS 检测到链路恢复时恢复链路。我们需要增加下面的函数，实现上面的功能：

```
1 @set_ev_cls(ofp_event.EventOFPPortStatus, MAIN_DISPATCHER)
2 def port_status_handler(self, ev):
3     msg = ev.msg
4     dp = msg.datapath
5     ofp = dp.ofproto
6     port_no = msg.desc.port_no
```

```

7
8 if msg.reason == ofp.OFPPR_MODIFY:
9     if msg.desc.state & ofp.OFPPS_LINK_DOWN:
10         # 链路故障
11         link = self.network_awareness.port_link.get((dp.id, port_no))
12         if link:
13             src_dpid, dst_dpid = link
14             if self.network_awareness.topo_map.has_edge(src_dpid, dst_dpid):
15                 self.network_awareness.topo_map.remove_edge(src_dpid, dst_dpid)
16                 self.logger.info('link down: %s -> %s, rerouting...',
17                                 src_dpid, dst_dpid)
18             self.sw = {}
19             for dpid, datapath in self.network_awareness.switch_info.items():
20                 self.delete_flow(datapath)
21         else:
22             # 链路恢复
23             link = self.network_awareness.port_link.get((dp.id, port_no))
24             if link:
25                 src_dpid, dst_dpid = link
26                 if not self.network_awareness.topo_map.has_edge(src_dpid, dst_dpid):
27                     self.network_awareness.topo_map.add_edge(
28                         src_dpid, dst_dpid, hop=1, delay=0)
29                     self.logger.info('link recovered: %s -> %s, rerouting...',
30                                     src_dpid, dst_dpid)
31             self.sw = {}
32             for dpid, datapath in self.network_awareness.switch_info.items():
33                 self.delete_flow(datapath)

```

其次，我们还要删除掉旧的流表，保证交换机正确转发。下面是 delete\_flow 函数，给上面的代码调用：

```

1 def delete_flow(self, datapath):
2     ofp = datapath.ofproto
3     parser = datapath.ofproto_parser
4     match = parser.OFPMatch()
5     mod = parser.OFPFlowMod(
6         datapath=datapath,
7         command=ofp.OFPFC_DELETE,
8         out_port=ofp.OFPP_ANY,
9         out_group=ofp.OFPG_ANY,
10        match=match
11    )
12    datapath.send_msg(mod)
13    # 重新下发 table=miss 流表项
14    actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER, ofp.OFPCML_NO_BUFFER)]
15    self.add_flow(datapath, 0, match, actions)

```

## 4.3 测试

我们看看 down 掉 S9 和 S8 之间的链路后，情况如何：

```

mininet> link s9 s8 down
mininet> SDC ping -c 5 MIT
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=76.5 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=149 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=156 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=162 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=168 ms

```

图 17: Time(Down)

可以看到，RTT 从 129ms 变到 150ms 左右，证明路径改变了。下面我们看看走的具体路径是哪一条：

```

path: 10.0.0.5 -> 10.0.0.3
10.0.0.5 -> 1:s6:3 -> 2:s7:3 -> 2:s8:1 -> 10.0.0.3

```

图 18: Route

我们可以与之前的路径对比，发现路径绕开了断掉的 S9-S8 这一条链路，逻辑正确。之后我们再把这条链路 UP，看一看情况如何：

```

mininet> link s9 s8 up
mininet> SDC ping -c 5 MIT
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=69.2 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=130 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=136 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=129 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=128 ms

```

图 19: Time(Up)

可以看到，RTT 已经恢复正常了，这证明现在的 controller 已经拥有了容忍链路故障的能力！